Building So-What Reports: Every Number Says What to Do Next

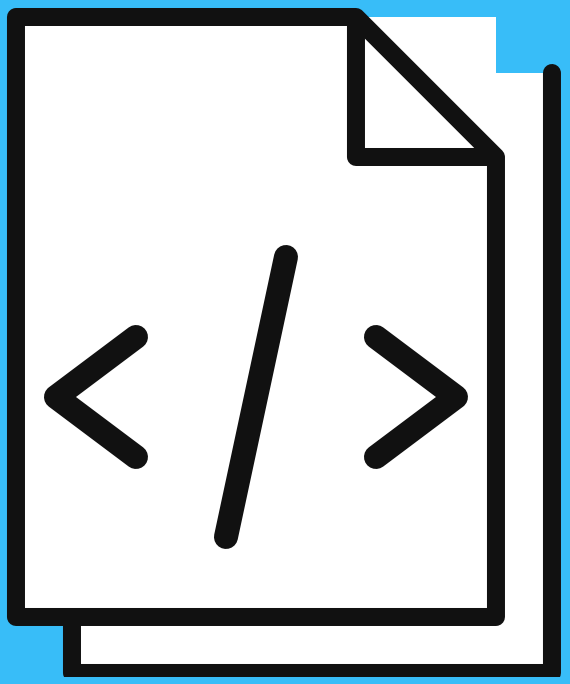
How one factory office stopped getting "okay, noted" and started getting decisions, from slide decks and dashboards built on 856,850 real operations. The problem, the pattern behind it, and where it goes next. November 2026.

A real case study by Ah Hock, The Industrial Scribe

One office in a factory in the semiconductor supply chain. Real counts from three working systems. Anonymised: no company, customer or colleague names, no amounts, no part numbers.

Contents

Executive summary	4
The problem and the use cases	6
Architecture pattern	8
Is it effective, and where is the room to improve?	12
Looking forward	15
Next steps	17
Appendix: The architecture behind it	19
Appendix: Results in numbers	21
Appendix: Glossary	23



Chapter 1

Executive summary

Chapter 1

Executive summary

The numbers were right. Nobody knew what to do with them.

Every factory office makes reports. Ours made plenty: lead-time decks for reviews, idle-lot spreadsheets for planning, a daily output compile for the morning call. The data in them was usually correct. The trouble was what happened after. A manager looked at a chart, said "okay, noted", and nothing moved, because the chart showed a number and left everyone to work out what it meant.

I found three things kept going wrong. Titles named a topic ("Lead time by customer") instead of saying what the data proved. Numbers mixed things that needed different actions: lots waiting by plan were counted as idle, next to lots that were really stuck. And some sentences in our decks were simply false, because they were typed once for one customer and copied into the next.

This guide describes the three reporting systems I rebuilt around one question, "so what must happen next?", and the rules they taught me. Scripts do every calculation, so a number never depends on someone's mood or a model's guess. Every title and every claim is computed from the data or deleted. Every page says how it is measured and what changed since last time. Anything nobody acts on is removed.

The results, in counts. A capacity and lead-time analysis turned 856,850 operations into 10 customer decks and 5 shop-level decks, and its method matched a reference report on 99.64% of 39,222 steps. More than 20 deck sentences that were typed-in and wrong for some customers are now computed. A production dashboard replaced 3 separate departmental spreadsheets with one shared view. When it started splitting "waiting by plan" from truly idle, average idle days went from about 43 to 26. That was a correction to the definition, not a performance gain, and the page says so. A floor report gives the planner 7 views built from their own sheets. A wall board nobody used was deleted, and targets were switched off when people stopped trusting them.

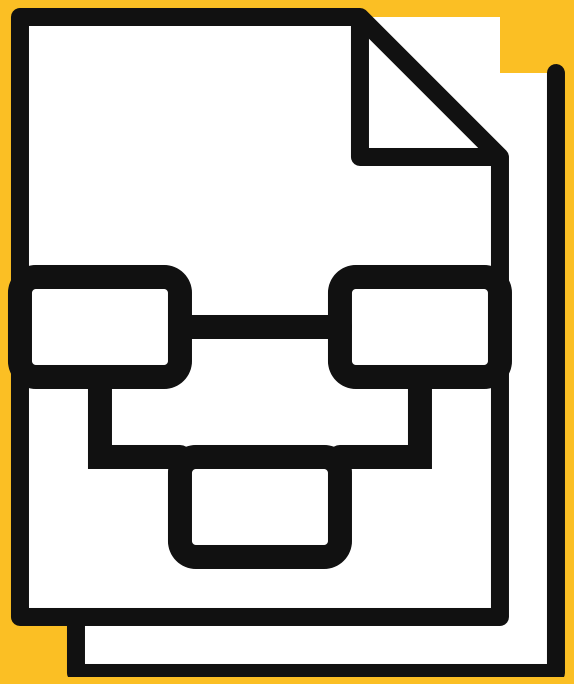
It is thin in two places, and I say so up front. How often a deck changed a decision has never been tracked. Time saved has never been measured. So this guide claims what the systems do, not what they earned.

The business case for scripts, with a check on every claim

A number has one right answer once the method is fixed, so scripts calculate everything. A model is never in the path that produces a number. Where a model earns its place is the check: reading every slide against its own numbers and catching a sentence that no longer holds. The decisions stay with the person presenting.

What this guide covers

- The problem, stated as situation, complication and resolution, and the use cases.
- The pattern: a script pipeline with a so-what layer on top, with a worked example.
- The eight rules these systems taught me, each from a real failure.
- Whether it is effective by the six tests, with evidence, and the method for finding the room to improve.
- What going higher means, what not to build, and one next decision.



Chapter 2

The problem and the use cases

Chapter 2

The problem and the use cases

Situation

The factory runs many customer groups on shared machine groups. Delivery dates are promised every week. Every lot, every routing step and every transfer time sits in the ERP (the factory's planning and records system), but only as exports: one of them close to 870,000 rows across fifteen sheets. Reports were built by hand from those exports, by different people, in different spreadsheets.

Complication

- **Topic titles.** "Delivery performance" tells the room what the chart is about, not what it proves. The audience has to do the presenting.
- **Mixed numbers.** Idle lots, lots waiting by plan and lots waiting for parts were added together. The total looked alarming and pointed at nobody.
- **Copied sentences.** Decks written for one customer carried fixed sentences into another customer's deck, where the data said something else.
- **Three versions of the truth.** Sales, planning and purchasing each kept their own spreadsheet. A customer's question got whichever answer the person on the phone had.
- **Screens people learn to ignore.** A target that showed zero against seventy taught the floor to stop looking.

Resolution

Scripts calculate. Rules written down in words decide what counts as what. Every title and claim comes from the data. Every page says how it is measured, when the definition changed, and what moved since last time. Whatever nobody acts on gets deleted.

Use cases

Customer review decks. For each customer group: how long a lot really takes, where the days go, and which parts never clear inside thirteen weeks. 10 customer decks, 24 slides each, plus a workbook per group.

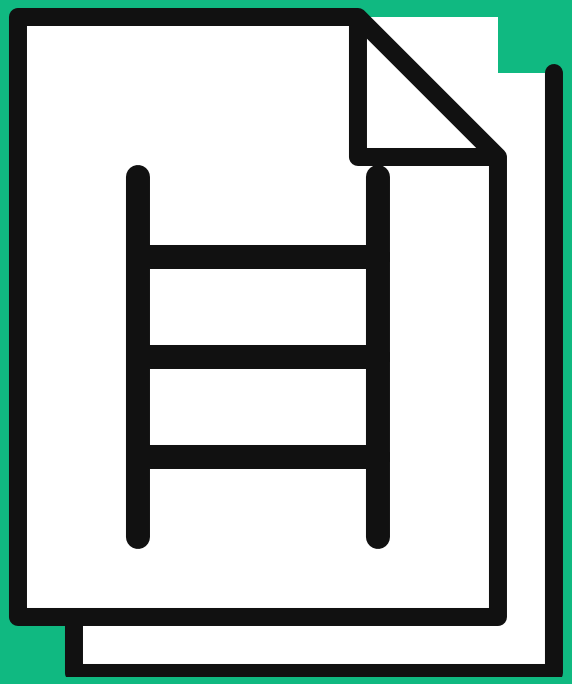
Shop-level decks. Machining hours by machine group, seven-year trends for 25 machine groups, and reviews of key machine groups, so a claim like "this process never got faster" can be checked before anyone says it in a meeting.

One shared production view. Every idle lot, why it is idle, what it waits for and what comes next, for sales, planning and purchasing on one page instead of three spreadsheets.

The daily deck. Built by a script from the same data each time a new export lands, showing what changed since yesterday rather than repeating what everyone already knows.

The planner's morning. Yesterday's target, completed, shortfall and carried; today's action plan with candidates from the overdue list and who runs them; what changed in the last sync; and how fresh each source sheet is.

A commercial slide. A cash and pre-payment proposal built as three parts, the reference point, the gap and the proposal, through many revisions, alongside the reply to the customer's director.



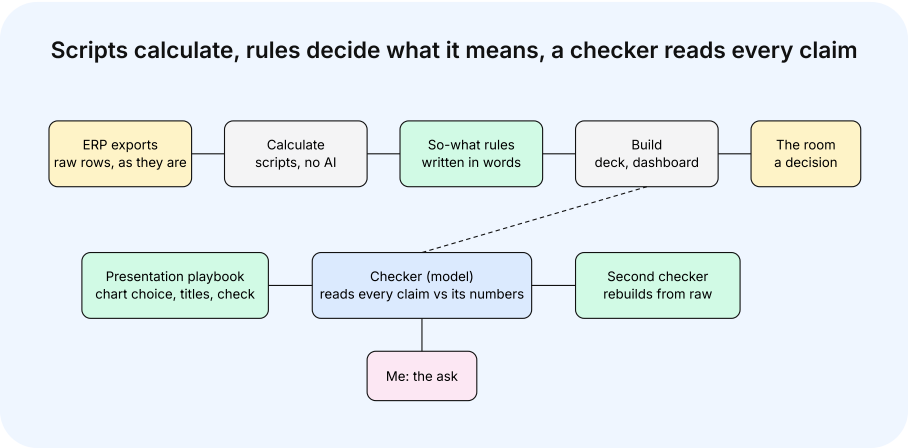
Chapter 3

Architecture pattern

Chapter 3

Architecture pattern

A script pipeline with a so-what layer on top



Architecture overview: a fixed chain carries raw exports to a finished deck or dashboard. Scripts do every sum, split and percentage. Between the numbers and the page sits a layer of rules written in plain words: what counts as idle, what counts as late, what title a number earns. A checker reads every built page against its own numbers before it reaches me, and on the biggest analysis a second checker rebuilds the key figures from the raw export with its own code. I decide the method and make the ask.

When to use: whenever a report exists to make someone decide or act, and the same report comes back every week or month.

When to avoid: putting a model in the path that produces a number. A number that changes with the model's mood is worse than no number. Also avoid it for one-off exploration, where the question isn't settled yet and a rule would freeze it too early.

Example: one number that was misleading the management

1. **The observation.** Assembly lots are launched early by design, so they sat in the idle count with huge idle days. Management read it as a problem. Lots whose start date had not arrived yet were counted as idle too.
2. **The rule, agreed in words.** A lot whose last possible start date has not arrived is waiting by plan and counts zero. An assembly lot missing a part is waiting for its kit, which is the debt of whoever supplies that part. Only a lot with its parts on hand and its date passed is truly idle. That is the number to act on.
3. **The build.** The dashboard shows the three groups separately and puts the truly idle number where people look first. Each tab has a "how this page is measured" panel.
4. **The check.** A sample is recomputed from the raw files and compared.
5. **The honest note.** Average idle days went from about 43 to 26 on 9 September. That is the definition changing, not the factory improving, so the page says so and trends across the change are not compared.

The eight rules, each from a real failure

Rule	Where it came from
1. Scripts calculate. Nothing is estimated by eye or by a model.	Repeated postings seconds apart look like separate jobs. A quick count doubles throughput and halves waiting time.
2. Every title and claim is computed from the data, or deleted.	More than 20 deck sentences were typed once, copied between customers, and wrong for some of them. All are now computed.
3. Show the number someone can act on, not the total.	Idle mixed three things. Split into waiting by plan, waiting for kit and truly idle, only one of them needed action.
4. When a definition changes, say so on the page.	Idle went from about 43 to 26 days by definition. Presented as a gain, it would have been a lie.
5. Never show a target nobody trusts.	Targets on the floor report were switched off at the first zero-against-seventy display. A target people don't believe teaches them to ignore the screen.
6. Delete what nobody uses.	A wall board looked right, was built, and was not used. It was removed.
7. Show what changed and how fresh it is.	The daily deck shows changes since yesterday; the planner sees a "what changed" panel after every sync and a freshness mark on each source sheet.
8. One shared view beats three private ones.	Sales, planning and purchasing each had a spreadsheet. Now they read one page with one set of rules.

How a report is built, step by step

Step	Who does it	What happens	What it produces
1. Collect	Script, no AI	Copies the newest exports from the shared drive	A run folder
2. Judge the input	The model	Compares the new export with the last one and checks the dates inside it	Go, or a stop and a question to me
3. Calculate	Script, no AI	Applies the written rules: visits, waits, idle groups, loads	One clean table of numbers
4. Title	Script, from the rules	Each title and claim is filled from the numbers, not typed	Takeaway titles that stay true
5. Build	Script, no AI	Decks, workbooks, dashboard tabs, with method and source on every page	Files
6. Check	The model	Reads every claim against its numbers, looks for stale wording	A defect list, then fixes
7. Verify	A second model	Rebuilds key figures from the raw export with its own code	Pass or fail per figure
8. Release	Me	Reads it, decides open method questions, makes the ask	A report in the room

Key challenge: inputs that look fresh but are not

The latest export carried a new date in its name but was the previous export with three months of rows pasted on top, overlapping, with a lost column header. Without a sheet-by-sheet comparison, stale statuses and duplicate rows would have gone into every figure on every slide. The fix in place: compare each new export with the last one and check the last timestamp inside it. The named next fix: a true full export each time.

Key challenge: the analyst's own assumptions

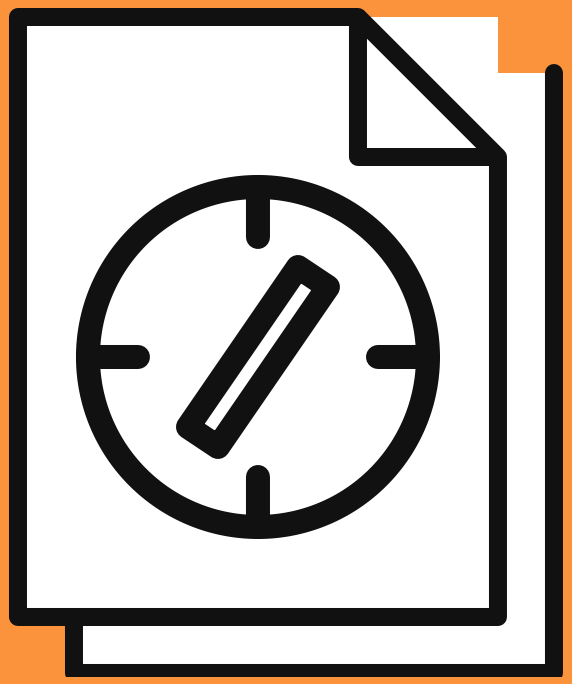
One load figure assumed all new machines were available from a single date. The updated machine list showed they arrived later, so the figure was withdrawn before anyone used it. The fix in place: the shared drive is the master and beats any assumption. A wrong number caught before the meeting costs nothing. After the meeting, it costs your credibility.

Design best practices, applied

Start simple. A script and a page. Each dashboard tab was added only when someone asked a real question. **Choose the right model for the job.** None in the path that makes numbers; the strongest model for checking claims; a second, independent one for verifying the biggest analysis. **Practice modular design.** Rules live in one written document; the original scripts stay untouched and one patch script records every change. **Build observable systems.** Every page states its period, source and method; every definition change is dated.

Decision framework for this module

Control: high. These numbers decide what runs next. **Domain complexity:** one domain, messy inputs. **Resources:** one laptop and the shared drive. **Expertise:** written rules and a presentation playbook, not one person's memory. **Step on the ladder:** step 0 for numbers; a draft-and-check pair for claims.



Chapter 4

Is it effective, and where is the room to improve?

Chapter 4

Is it effective, and where is the room to improve?

The guide never defines "effective" in one sentence. It defines an agent as a model that directs its own steps toward a goal, and lets "effective" emerge from six tests across its chapters. These reports are deliberately not agents where numbers are made, so the first test is decided by refusing one.

The six tests, with evidence

Test	What it asks	Evidence on the so-what reports	Verdict
1. Right problem for an agent	Is the path unknown in advance? If not, a script is better.	Every number has one right answer, so scripts make them. Only the claim check is open-ended, and that is where the model sits.	Pass
2. Simplest design that works	Started small, added only on need?	Each tab and deck added when someone asked; the second checker added after defects slipped through; a wall board removed when unused.	Pass
3. Matched to control, domain, budget, expertise	Answered honestly?	High control, scripts in the number path; one domain; one laptop; rules written down.	Pass, with one laptop as the weak point
4. Observable	Can we see why a number is what it is?	"How this page is measured" on every tab; definition changes dated; a what-changed panel and freshness marks.	Pass
5. Modular	Parts change without redesign?	Parsers read headers, so they survived two format changes; rules change in one place.	Pass
6. One metric tied to a business outcome	One number, moving?	Truly idle days and lead time per customer are measured. Whether a deck's ask got a yes is not tracked.	Partial

Why it counts as effective. Tests 1 and 4 carry it. No number depends on a model, and every number can be traced to a written rule and a dated definition. That is what lets a manager trust a takeaway title enough to act on it. The honest weakness is test 6: the systems produce decisions-ready reports, but nobody has counted how many decisions they produced.

How to look for the room to improve

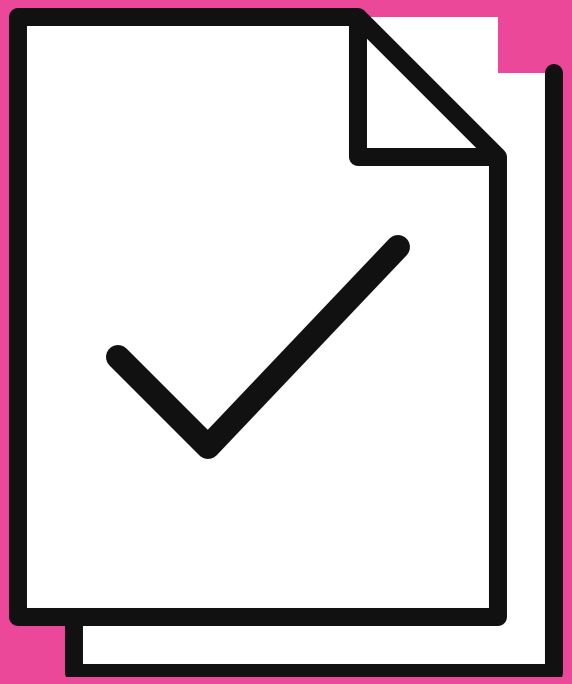
1. **Re-ask the four decision questions.** The reports now serve a whole team, so one laptop is no longer an acceptable home for them.
2. **Read "when to avoid" against actual use.** No model makes numbers. The remaining risk is typed-in text: one 37-slide shop-wide deck still carries typed figures.
3. **Compare the plan with what actually ran.** The plan says one fresh export per refresh. What actually ran was a pasted export once.
4. **Watch the metric.** Lead time is measured every refresh but not yet set against promised dates.
5. **Reopen every key-challenge box.** Fresh-looking inputs: caught by checking, removed only by a full export. Own assumptions: the shared drive now wins.

6. **Check the ladder before climbing it.** The draft-and-check pair already exists. Nothing needs a new agent.
7. **Check reach.** The floor report's adoption is not measured yet, and the analysis cannot see promised dates or actual run times.

The room to improve, found by that method

Lever	Finding	Size
Decision questions	One laptop hosts reports a team relies on	Large
When to avoid	One 37-slide deck still has typed-in figures	Medium
Plan vs what ran	Exports are pasted, not freshly pulled	Medium
Metric	Nobody counts whether the ask got a yes	Large
Reach	Promised dates and floor adoption not measured	Large

Chapter 5 orders these into what to build next.



Chapter 5

Looking forward

Chapter 5

Looking forward

What going higher means for so-what reports

1. **Count the asks.** Every deck ends with one ask and a date. Log the answer each time: yes, no or later. That turns "effective" from a feeling into a number.
2. **An act-today band on every dashboard.** The planner's morning view already lists today's candidates and who runs them. The same idea belongs at the top of every dashboard: at most three items that crossed a written rule, each with the action, the owner and the date.
3. **Finish the last typed deck.** Move the 37-slide shop-wide deck onto computed figures, so no claim anywhere is typed.
4. **Targets back on, when trusted.** Switch floor targets back on once the plan matches the floor for a few weeks running, not before.
5. **Off the laptop.** An always-on home for the reports, so a closed laptop stops the morning picture.

What not to do. Never put a model in the path that makes numbers. Never add a tab or a slide nobody asked for. Never compare trends across a definition change. Never switch on a target just to make the page look complete.

Measure of success. The share of asks that get a decision, yes or no, in the meeting they are made.



Chapter 6

Next steps

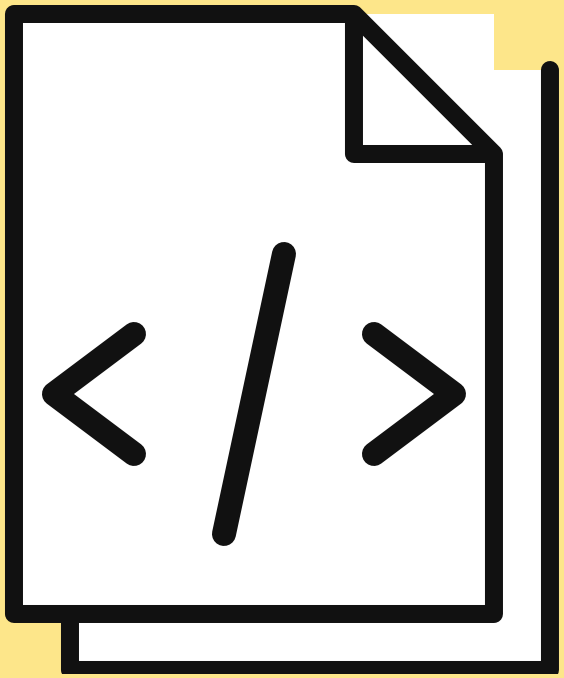
Chapter 6

Next steps

Decision, yes or no: for the next four weekly reviews, log every ask and its answer (yes, no or later), and compare with the four reviews before.

If you want reports like this

You don't need the scripts or the laptop to start. The Industrial Scribe has packaged the method as a kit, the "Okay, Noted." Fix. Paste your data into any AI chat and you get slides where every title says what it proves, plus a dashboard with an act-today band, a takeaway title and an action line on every tile, and colour only where action is needed. If you use Claude Code, it does the maths with a script and builds the real deck or dashboard file for you. It never invents a target, an owner or a cause. It will be listed in the shop at theindustrialscribe.com.



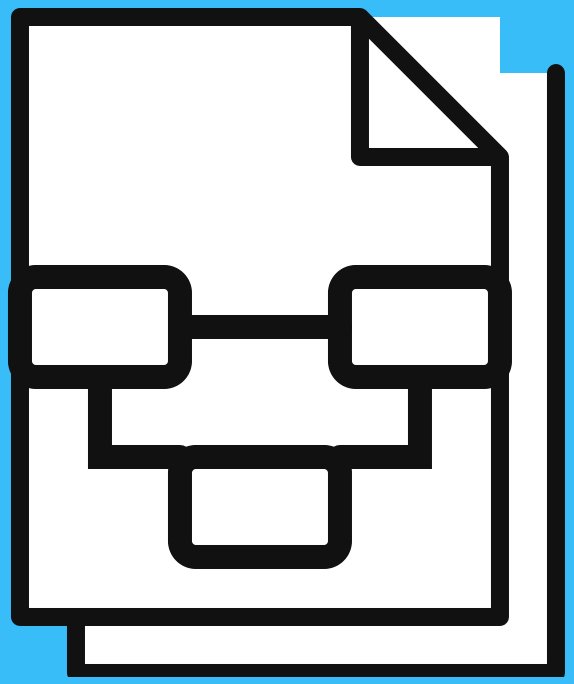
Appendix

Appendix: The architecture behind it

Appendix: The architecture behind it

System	What goes in	What scripts do	Where a model is used	What comes out
Capacity and lead-time analysis	A raw work-in-progress export (about 870,000 rows, 15 sheets), the routing, the machine list	Merge, collapse repeated postings into visits, charge each day to the step that held it, measure load against real machine counts	Judging each new export; checking every deck claim; a second model rebuilding key figures	Customer and shop-level decks and workbooks
Production dashboard	Work-in-progress, sales orders, bill of materials (about 252,000 rows), purchasing status, daily stock balance	Pick the newest files, split idle into three groups, resolve each lot's next one to three processes, build the daily deck	None when it runs; a developer agent with a checker when the logic changes	One shared page with tabs, a daily deck, a read-only cloud copy
Floor report	A machine QR scan and three operator taps; the planner's own sheets	Time-stamp on the server, compile yesterday on a closed eight-to-eight cycle, build today's plan, queues and a 21-working-day schedule	None when it runs; used to build and test every screen	An operator screen and a planner dashboard with 7 views

Boundaries. Read-only on the ERP and the shared drive. Nothing is written to the shared drive unless I ask. Data stays on company machines and the company network. Reports are never sent by the system; I take them into the room.



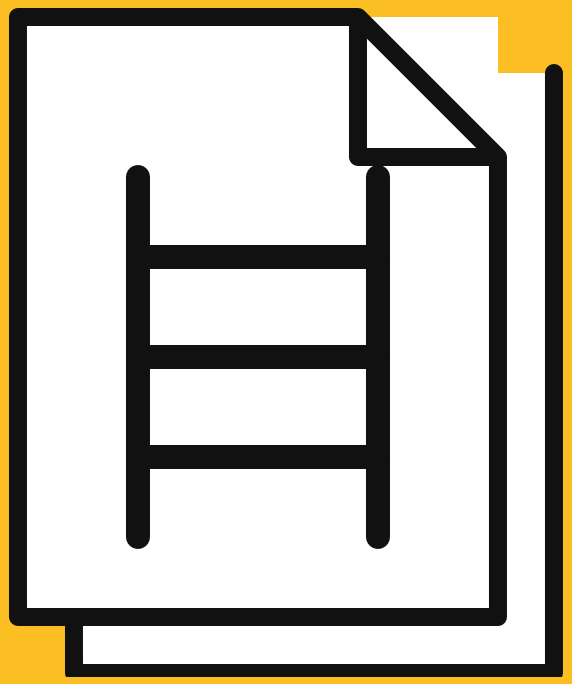
Appendix

Appendix: Results in numbers

Appendix: Results in numbers

Counts only, from the systems' own notes and run records, refreshed to 31 October 2026. These are real working systems run by one person in a real factory office, not demos. They show what the method can do in one office, not what every office will get. Where they fell short, this guide says so.

Measure	Value
Raw export rows parsed for the capacity analysis	869,469, merged to 856,850 operations
Lots behind the analysis / finished lots analysed across 7 customer groups	159,766 / 13,080
Customer decks / shop-level decks / workbooks	10 / 5 / 9
Slides per customer deck	24
Standard-hour method matched to a reference report	99.64% of 39,222 steps
Independent verifier passes on the biggest analysis	2
Deck sentences that were typed, wrong for some customers, now computed	over 20
Departmental spreadsheets replaced by one shared production view	3
Average idle days before and after splitting out "waiting by plan" (9 Sep 2026)	about 43, then 26. A change of definition, stated on the page, not a gain
Planner views compiled from the planner's own sheets	7
Operator taps per floor report / fields typed	3 / 0
Features removed because nobody used them	1 (the wall board). Targets on hold until trusted
Still typed-in, not yet computed	one 37-slide shop-wide deck
Time saved / decisions changed by a deck	not measured, so not claimed



Appendix

Appendix: Glossary

Appendix: Glossary

So what. The decision or action a number should trigger. The question every title, slide and tile must answer.

Takeaway title. A title that states what the data proves ("On-time delivery recovered to 96%"), not a topic ("Delivery performance").

ERP. The factory's planning and records system. It holds every order, lot and step, but reports come out of it only as exports.

Lot. One batch of one part, moving through its routing steps.

Waiting by plan. A lot whose last possible start date has not arrived. Not idle, so it counts zero.

Waiting for kit. An assembly lot missing at least one part. The debt of whoever supplies the part, not the assembly team.

Truly idle. A lot with its parts on hand and its start date passed, not running. The number to act on.

Lead time. Calendar days from a lot's first operation to finished.

Load. Standard hours arriving at a machine group, divided by the hours its machines can offer.

Checker, verifier. A model that reads a built report against its numbers; a second model that rebuilds key figures from the raw data with its own code and tries to prove them wrong.

Draft-and-check. One step produces, another checks against set criteria. The guide calls it evaluator-optimizer.

Act-today band. The strip at the top of a so-what dashboard: at most three items that crossed a written rule, each with the action, the owner and the date.

Step on the ladder. How much freedom a design gives the model, from step 0 (scripts only) to step 4 (peer agents).